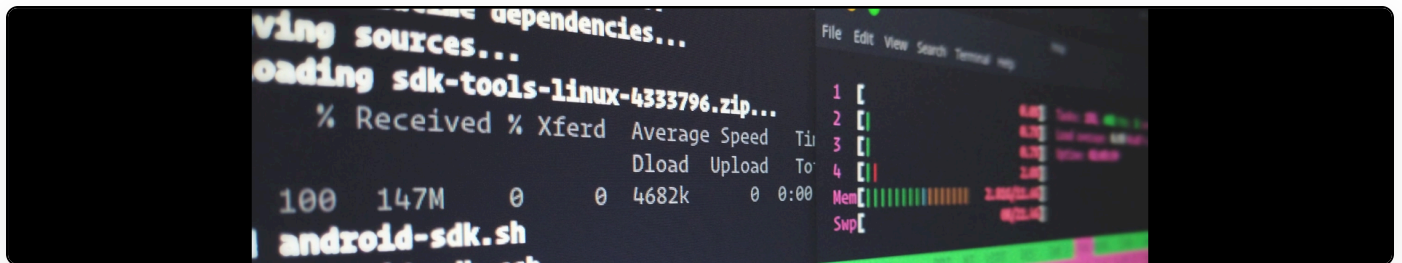


Using Django ORM, Click, and Rich to create useful command-line python apps

Cite as: Martin Paul Eve, "Using Django ORM, Click, and Rich to create useful command-line python apps", *eve.gd: Martin Paul Eve*, January 24, 2022, <https://doi.org/10.59348/pvsyt-hbv27>.



This is a post to document the setup that I use when I want quickly to create a great functional command-line python application with ORM database support.

Django ORM

The Django ORM is an incredibly powerful database tool that lets you design models in python and then persist them to the database. Django, though, is primarily a web framework. I don't want that in a command-line app, but it is possible to use the ORM without other components.

The way to do this is to have a structure as follows:

```
Project root
--> main.py
--> app_dir
--> --> settings.py
--> --> models.py
--> --> \_\_init\_\_.py
```

Inside main.py, you need the following:

```
import sys
import inspect
import os.path

sys.path.append(os.path.dirname(os.path.abspath(inspect.getfile(inspect.currentframe()))))
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'app_dir.settings')

import django
django.setup()
```

This will initialize the Django application.

You can also, from this point on, use the manage.py commands to setup the database and do everything that you normally would in Django. But you can also access your models and the ORM from the main.py file.

Click

Parsing commandline arguments is tedious and I don't want to spend time doing it. This is where the brilliant Click comes in. Click will handle the arguments and pass them to your function.

My usual setup for Click is something like this:

```

@click.group()
def cli():
    pass

@click.command()
@click.option('--url',
              prompt='URL',
              help='The URL to fetch')

@click.option('--cache/--no-cache',
              help='Whether to cache or not',
              default=True)
def import_single(url, cache):
    pass

if __name__ == '__main__':
    cli.add_command(import_single)
    cli()

```

To add additional commands, you just add them to the cli object in the main function. This allows me just to design functions as I need them and have automatic wire-up to the CLI.

Rich

I only discovered this just the other day, but the [Rich library](#) provides loads of really beautiful functionality for command-line applications to create better interactivity and display.

For instance, with just a few lines of code, you can reconfigure the python logger to use Rich for output:

```

from rich import pretty
from rich.logging import RichHandler

FORMAT = "%(message)s"
logging.basicConfig(
    level="NOTSET", format=FORMAT, datefmt="[%X]", handlers=[RichHandler()])

log = logging.getLogger("rich")

```

Now, I can log whatever I want using lines like:

```
log.info('[green]Data written to out file:[/] {}'.format(out_file), extra={'markup': True})
```

and my application will display a neat timestamped progression, with python file and line numbers in the right-hand column.

With apologies to [tqdm](#), the progress bar stuff in Rich looks even neater than that great library, so I'll be using that in future.

So these are just a few of the basic tools and configurations that I use for my personal projects.